PRÁCTICA 1: SISTEMAS OPERATIVOS. Gestión de procesos y archivos-

Marcos Sánchez García

ÍNDICE

1. Ejercicio malla.c…….pág 2
- 1.1. Enunciado…….pág 2
- 1.2. Tareas y justificación……...pág 2
- 1.3. Análisis del código……...pág 3

2. Ejercicio ejec.c…….pág 7
- 2.1. Enunciado…….pág 7
- 2.2. Ejemplo de salida por pantalla.....pág 7
- 2.3. Tareas y justificación……...pág 8
- 2.4. Análisis del código……...pág 8

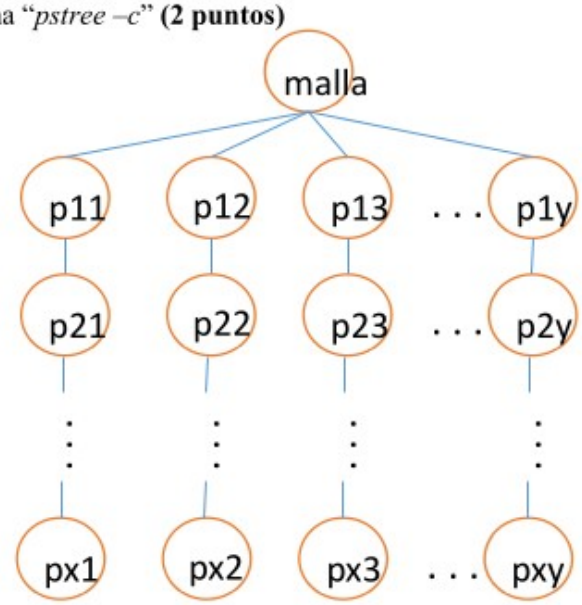
3. Ejercicio hacha.c……pág 12
- 3.1. Enunciado…….pág 12
- 3.2. Ejemplo de salida por pantal..…….pág 12
- 3.3. Tareas y justificación……...pág 12
- 3.4. Análisis del código……pág 13

4. Ejercicio hijos.c…….pág 17
- 4.1. Enunciado y salida esperada…….pág 17
- 4.2. Tareas y justificación……...pág 17
- 4.3. Análisis del código……...pág 18

1. Ejercicio: malla.c

1.1. Enunciado

a) Realiza un programa llamado **malla.c** que produzca el siguiente árbol de procesos. El programa recibirá dos argumentos 'x' e 'y' que representan el número de filas y columnas. Para comprobar la estructura de procesos que realmente estamos creando debemos emplear la llamada al sistema "*pstree –c*" **(2 puntos)**



1. 2. Tareas y justificación

- **Validar argumentos**: Necesitamos los argumentos x e y para un correcto funcionamiento. Sino, lanza el mensaje y termina el programa.
- **Crear el árbol**: Controlamos que inicialmente, cada columna, tenga un hijo y luego cada una de ellas tiene más hijos hasta completar las ramificaciones llegando al hijo final.
- **Esperar sin** sleep: el padre usa alarm(1) y pause() para dar tiempo a que se cree todo el árbol.
- **Mostrar el árbol**: el padre crea un hijo que hace execlp de pstree.
- **Enviar la señal**: el padre manda una señal a todos (SIGUSR1) . Pero solo necesitamos que la ejecuten las hojas finales y los procesos intermedios las ignoren (usamos SIG_IGN).
- **Destrucción controlada**: las hojas mueren, los intermedios esperan a su hijo con wait() y luego mueren, y el padre espera a todas las columnas y muere el último.

1.3. Análisis del código

main()

Es el programa principal y el que organiza la funcionalidad del código al completo. Hace tres cosas, en este orden:

1. **Comprueba los argumentos.** Si argc no es 3 (el nombre del programa más x e y), imprime “Se necesita el número de filas y columnas” y sale con exit(1). Si son correctos, convierte x e y a entero con atoi().
2. **Prepara las señales.** Hace signal(SIGUSR1, SIG_IGN) antes de crear ningún hijo. Esto es importante: los hijos heredan esta configuración al hacer fork(), así que todos los procesos ignoran SIGUSR1. Más adelante solo las hojas cambiarán esto instalando su propio manejador.
3. **Crea el árbol y reparte los papeles.**
    - Llama a crearColumnas(y) y guarda el resultado en pid_y. Después de esta llamada hay y procesos con pid_y == 0 (los hijos) y el padre original con pid_y != 0.
    - Llama a crearFilas(x, pid_y) y guarda el resultado en j. Solo los hijos hacen fork() dentro, el padre pasa de largo.
    - Según el valor de las variables, cada proceso toma un camino distinto:
        - pid_y == 0 y j == x-1: es una hoja, ejecuta comportamientoHoja().
        - pid_y == 0 y j != x-1: es un intermedio, ejecuta comportamientoIntermedio().
        - pid_y != 0: es el **padre**, ejecuta comportamientoPadre(y).

```c
int main(int argc, char* argv[]){
    int x,y,j;
    pid_t pid_y;

    if(argc!=3){
        printf("Se necesita el número de filas y columnas\n");
        exit(1);
    }

    x=atoi(argv[1]);
    y=atoi(argv[2]);

    signal(SIGUSR1, SIG_IGN);     // Para que solo hagan SIGUSR1 las hojas.

    pid_y=crearColumnas(y);
    j=crearFilas(x, pid_y);

    if(pid_y==0){
        if(j==x-1){
            comportamientoHoja();
        }
        else{
            comportamientoIntermedio();
        }
    }
    else{
        comportamientoPadre(y);
    }
    return 0;
}
```

manejador()

Función vacía. Solo sirve para que pause() se desbloquee cuando llega una señal.

crearColumnas(y)

Bucle con fork() que crea y hijos. Cada hijo sale del bucle con break para no crear más columnas. Devuelve pid_y, que vale 0 en los hijos.

Llamadas: fork().

```
pid_t crearColumnas(int y){
    int i;
    pid_t pid_y = -1;

    for(i=0;i<y;i++){
        pid_y=fork();
        if(pid_y==0){
            break;
        }
    }
    return pid_y;
}
```

crearFilas(x, pid_y)

Solo la ejecutan los hijos. Cada uno hace fork(), el padre sale del bucle y el nuevo hijo sigue. Así se forma la cadena. Devuelve j, que vale x-1 en la hoja.

Llamadas: fork().

```
int crearFilas(int x, pid_t pid_y){
    int j;
    pid_t pid_x=-1;

    for(j=0;j<x-1;j++){
        if(pid_y==0){
            pid_x=fork();
            if(pid_x!=0){
                break;
            }
        }
    }
    return j;
}
```

comportamientoHoja()

Instala el manejador de SIGUSR1, que solo se instala en las hojas, y se queda en pause(). Al recibir la señal imprime “Hoja muere” y termina.

Llamadas: signal(), pause(), getpid(), exit().

comportamientoIntermedio()

Hace wait() hasta que muere su hijo y entonces imprime “Intermedio muere”. Como ignora SIGUSR1, la señal no le interrumpe el wait.

Llamadas: wait(), getpid(), exit().

```c
void comportamientoHoja(void){
    signal(SIGUSR1, manejador);   // la hoja sí reacciona a la señal
    pause();
    printf("Hoja %d muere\n", getpid());
    exit(0);
}

void comportamientoIntermedio(void){
    wait(NULL);                   // SIG_IGN no interrumpe el wait
    printf("Intermedio %d muere\n", getpid());
    exit(0);
}
```

mostrarArbol()

Guarda el pid del padre en una cadena con sprintf, crea un hijo y ese hijo ejecuta pstree -c <pid>. El padre espera a que acabe.

Llamadas: fork(), execlp(), wait(), getpid().

```c
void mostrarArbol(void){
    char cadPadre[20];
    pid_t pid_cmd;

    sprintf(cadPadre, "%d", getpid());
    pid_cmd = fork();
    if(pid_cmd == 0){
        execlp("pstree", "pstree", "-c", cadPadre, NULL);
        exit(1);                  // solo si execlp falla
    }
    wait(NULL);                   // espera a que acabe el pstree
}
```

comportamientoPadre(y)

Hace alarm(1) y pause() para esperar un segundo, muestra el árbol, manda SIGUSR1 al grupo, espera a sus y hijos y imprime que muere el último.

Llamadas: signal(), alarm(), pause(), kill(), wait().

```c
void comportamientoPadre(int y){
    int i;

    signal(SIGALRM, manejador);
    alarm(1);
    pause();                    // da tiempo a crear el árbol

    mostrarArbol();

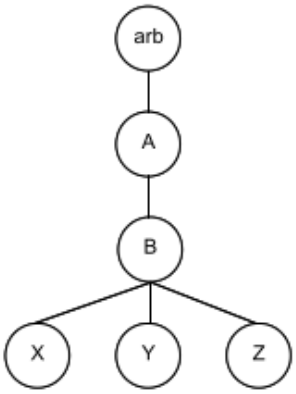
    kill(0, SIGUSR1);           // señal a todo el grupo: solo las hojas
                                 //(último proceso de la rama)la tratan

    for(i=0;i<y;i++){
        wait(NULL);
    }
    printf("Padre %d muere el último\n", getpid());
}
```

2. Ejercicio ejec.c

2.1. Enunciado

b) Realiza un programa llamado **ejec.c** que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando "pstree". El proceso Z no puede utilizar el comando "sleep", por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos). **(2 puntos)**



2.2. Ejemplo de salida por pantalla

El programa debe mostrar por la salida estándar la siguiente información:

```
$ ejec 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
/*Tras un intervalo de 15 segundos aparecerá*/
/*Resultado del comando "pstree" */
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

2. 3. Tareas y justificación

- **Comprobar argumentos**: Como el anterior ejercicio debemos asegurarnos que el usuario introduce los argumentos pertinentes.
- **Crear el árbol**: Diseñamos un algoritmo con bucles para que cada hijo tenga a su otro hijo y así sucesivamente.
- **Esperar sin** sleep: Z usa alarm(tiempo) y pause(). Cuando pasa el tiempo llega SIGALRM y pause() se desbloquea.
- **Dar la orden**: Cuando es debido, el proceso Z manda una señal a A para que muestre el árbol y comience a destruirlo comenzando por abajo.
- **Pasar a la destrucción**: cuando termina de mostrarse el árbol por pantalla, A manda una señal al proceso ejec, (el superpadre) que reacciona mandando otra señal distinta a A.
- **Destrucción controlada**: Dicha seññal baja por la cadena: ejec -> A -> B. B mata a X, Y y Z de uno en uno esperando a cada uno. Después mueren B, A y ejec, en ese orden.

2. 4. Análisis del código

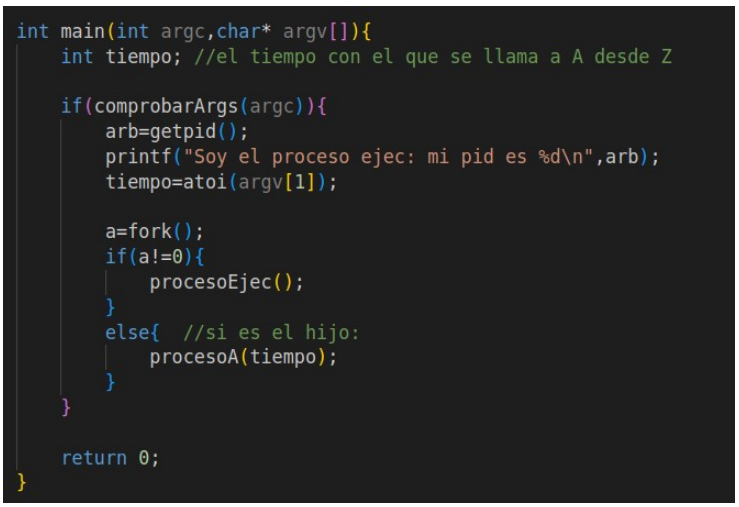
main()

Es el punto de entrada y reparte el trabajo. Hace tres cosas:

1. **Comprueba los argumentos** llamando a comprobarArgs(argc). Si no son correctos, no hace nada más.
2. **Guarda datos**: arb=getpid() para que todos sepan el pid de ejec, imprime su mensaje y convierte el argumento con atoi().
3. **Crea a A con** fork() y separa los caminos: el padre ejecuta procesoEjec() y el hijo ejecuta procesoA(tiempo).

Como los pid están en variables globales, los hijos los heredan al hacer fork().

Llamadas: getpid(), atoi(), fork().



```c
int main(int argc,char* argv[]){
    int tiempo; //el tiempo con el que se llama a A desde Z

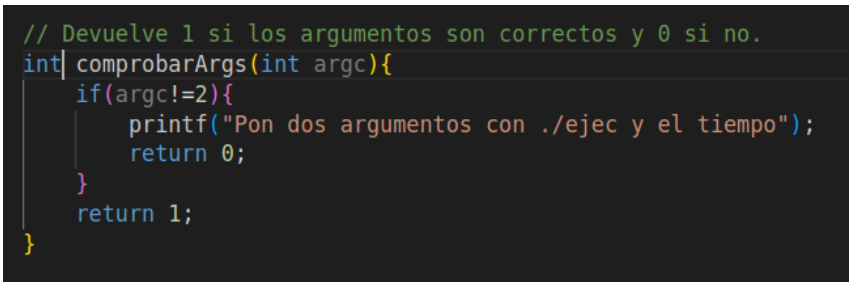
    if(comprobarArgs(argc)){
        arb=getpid();
        printf("Soy el proceso ejec: mi pid es %d\n",arb);
        tiempo=atoi(argv[1]);

        a=fork();
        if(a!=0){
            procesoEjec();
        }
        else{  //si es el hijo:
            procesoA(tiempo);
        }
    }

    return 0;
}
```

comprobarArgs(argc)

Mira si argc es 2 (el programa y el tiempo). Devuelve 1 si es correcto y 0 si no, y en ese caso imprime el aviso.

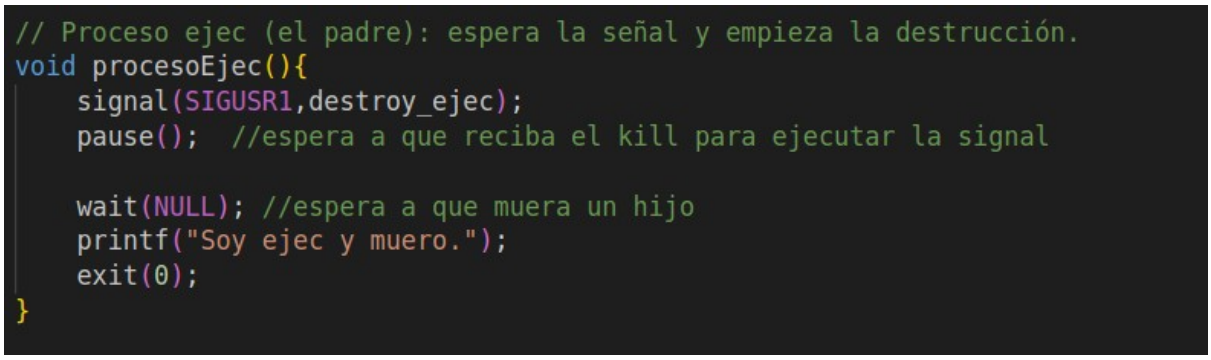


```c
// Devuelve 1 si los argumentos son correctos y 0 si no.
int comprobarArgs(int argc){
    if(argc!=2){
        printf("Pon dos argumentos con ./ejec y el tiempo");
        return 0;
    }
    return 1;
}
```

procesoEjec()

Es el proceso padre de todo. Instala destroy_ejec para SIGUSR1, se queda en pause() y, cuando llega la señal, hace wait(NULL) para esperar a A. Solo después imprime que muere y hace exit(0), así que es el último.

Llamadas: signal(), pause(), wait(), exit().



```c
// Proceso ejec (el padre): espera la señal y empieza la destrucción.
void procesoEjec(){
    signal(SIGUSR1,destroy_ejec);
    pause();  //espera a que reciba el kill para ejecutar la signal

    wait(NULL); //espera a que muera un hijo
    printf("Soy ejec y muero.");
    exit(0);
}
```

procesoA(tiempo)

Guarda su pid, imprime su mensaje y crea a B con fork(). Si es el padre (A), instala dos manejadores: pstree para SIGUSR1 y destroy_A para SIGUSR2. Luego espera con pause(), hace wait() a B y muere. Si es el hijo, ejecuta el proceso.

Llamadas: getpid(), fork(), signal(), pause(), wait(), exit().

```c
// Proceso A: crea B y propaga la destrucción.
void procesoA(int tiempo){
    a=getpid();  //el pid de A
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n",a,arb);
    b=fork();

    if(b!=0){
        signal(SIGUSR1,pstree);
        signal(SIGUSR2,destroy_A);
        pause();
        wait(NULL);
        printf("Soy A y muero\n");
        exit(0);
    }
    else{
        procesoB(tiempo);
    }
}
```

procesoB(tiempo)

Guarda su pid, imprime su mensaje y crea a X, Y y Z llamando a run_processX(), run_processY() y run_processZ(tiempo). Después instala el manejador vacío para SIGUSR2 y se queda en pause(). Cuando le llega la señal de A, manda SIGUSR2 primero a X, espera con wait(), luego a Y, espera, y luego a Z, espera. Así sus hijos mueren antes que él.

Llamadas: getpid(), signal(), pause(), kill(), wait(), exit().

```c
void procesoB(int tiempo){
    b=getpid();
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d.Mi abuelo es %d\n",b,a,arb);

    //Ahora hago x, y z de una forma en la que gracias al pause al pause() y al exit(0)
    // se pueden hacer correctamente las ramificaciones qie pide el eninciado.

    run_processX();
    run_processY();
    run_processZ(tiempo);

    signal(SIGUSR2,vacio);
    pause();            // aqui llega desde destroy A. Importante.
    kill(x,SIGUSR2);
    wait(NULL);
    kill(y,SIGUSR2);
    wait(NULL);
    kill(z,SIGUSR2);
    wait(NULL);
    printf("Soy B y muero\n");
    exit(0);
}
```

run_processX() y run_processY()

Cada uno hace fork(). El hijo imprime sus pid, instala el manejador vacío para SIGUSR2, se queda en pause() y al recibir la señal imprime que muere y hace exit(0). El exit(0) evita que el hijo siga ejecutando el código de B.

Llamadas: fork(), getpid(), signal(), pause(), exit().

```c
void run_processX(){
    x=fork();
    if(x==0){
        printf("Soy el proceso X mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo %d\n", getpid(), b, a, arb);
        signal(SIGUSR2,vacio);
        pause();
        printf("Soy X y muero\n");
        exit(0);
    }
}

void run_processY(){
    y=fork();
    if(y==0){
        printf("Soy el proceso Y mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo %d\n", getpid(), b, a, arb);
        signal(SIGUSR2,vacio);
        pause();
        printf("Soy Y y muero\n");
        exit(0);
    }
}
```

run_processZ(tiempo)

Igual que X e Y, pero antes de esperar la destrucción hace alarm(tiempo) y pause(). Al despertar manda SIGUSR1 a A con kill(), que es la orden del enunciado. Después se queda otra vez en pause() esperando SIGUSR2 y muere.

Llamadas: fork(), signal(), alarm(), pause(), kill(), exit().

```c
void run_processZ(int tiempo){
    z=fork();
    if(z==0){
        printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi abuelo es %d. Mi bisabuelo %d\n", getpid(), b, a, arb);
        signal(SIGALRM,vacio);
        alarm(tiempo);
        pause();
        kill(a,SIGUSR1); // llamo a a para que ejecute el pstree y empieze con la destrucción controlada del árbol
        signal(SIGUSR2,vacio);
        pause();
        printf("Soy Z y muero\n");
        exit(0);
    }
}
```

Manejadores

- vacio(): no hace nada. Solo sirve para que pause() se desbloquee al llegar la señal.
- pstree(): se ejecuta en A. Crea un hijo que hace execlp("pstree", ...). A espera con wait() a que termine, manda SIGUSR1 a ejec para que empiece la destrucción y se queda en pause().
- destroy_ejec(): se ejecuta en ejec. Manda SIGUSR2 a A.
- destroy_A(): se ejecuta en A. Manda SIGUSR2 a B.

Llamadas: fork(), execlp(), wait(), kill(), pause(), sprintf().

3. Ejercicio hacha.c

3.1. Enunciado

Ejercicio 2. Comunicación entre procesos: tuberías. (**3 puntos**)

Realizar un programa llamado **hacha.c** que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, … . Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <archivo> <tamaño>
<archivo> nombre del archivo a dividir
<tamaño> tamaño en bytes de los archivos divididos

- Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

3.2. Ejemplo de salida esperada:

```
$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02
```

3.3. Tareas y justificación

- **Comprobar argumentos**: hacen falta dos (fichero y tamaño del trozo). Si no, se muestra el uso que se debe emplear de los argumentos y termina.
- **Abrir el fichero**: el padre lo abre en modo lectura. Si no existe, avisa y termina.
- **Calcular los trozos**: con stat() se obtiene el tamaño del fichero. Se divide entre el tamaño del trozo y, si sobran bytes, hace falta un trozo más.
- **Crear un hijo por trozo**: en cada vuelta se crea una tubería nueva y se hace un proceso nuevo.
- **Pasar los datos**: el padre lee un trozo del fichero y lo escribe en la tubería. El hijo lo lee de la tubería y lo escribe en su fichero.

- **Esperar a los hijos**: el padre espera tantas veces como trozos haya, para que no termine antes que ellos.

3.4. Análisis del código

main()

Comprueba que argc sea 3. Si no lo es, imprime cómo se usa el programa. Si es correcto, llama a partir() con el nombre del fichero y el tamaño del trozo, que convierte con atoi().

Llamadas: atoi().

```c
int main(int argc, char *argv[]){
    if(argc != 3){
        printf("Error. Uso: %s fichero tam_trozo\n", argv[0]);
    }
    else{
        partir(argv[1], atoi(argv[2]));
    }
    return 0;
}
```

partir(nombre, tamanyo)

Es la función principal. Reserva con malloc el buffer donde el padre guarda cada trozo leído, y abre el fichero con open(). Si falla, imprime el error. Si va bien, calcula cuántos trozos hay con calcularTotalTrozos() y los crea con crearTrozos(). Cuando el proceso que vuelve es el padre (i == totalTrozos), libera el buffer y espera a los hijos con esperarHijos().

Llamadas: malloc(), open(), free().

```c
void partir(char nombre[], int tamanyo){
    int i;
    int entrada;
    int totalTrozos;
    char *buffer;

    buffer = (char *) malloc(sizeof(char) * tamanyo);
    entrada = open(nombre, O_RDONLY);
    if(entrada < 0){
        printf("Error. No existe el fichero\n");
    }
    else{
        totalTrozos = calcularTotalTrozos(nombre, tamanyo);
        i = crearTrozos(entrada, nombre, totalTrozos, buffer, tamanyo);
        if(i == totalTrozos){
            free(buffer);
            // bucle en el que el padre espera a los hijos.
            esperarHijos(totalTrozos);
        }
    }
}
```

calcularTotalTrozos(nombre, tamanyo)

Usa stat() para rellenar la estructura con la información del fichero y saca el tamaño con st_size. Divide entre tamanyo. Si el resto no es 0, suma uno, porque el último trozo será más pequeño. Devuelve el total.

Llamadas: stat().

```c
void partir(char nombre[], int tamanyo){
    int i;
    int entrada;
    int totalTrozos;
    char *buffer;

    buffer = (char *) malloc(sizeof(char) * tamanyo);
    entrada = open(nombre, O_RDONLY);
    if(entrada < 0){
        printf("Error. No existe el fichero\n");
    }
    else{
        totalTrozos = calcularTotalTrozos(nombre, tamanyo);
        i = crearTrozos(entrada, nombre, totalTrozos, buffer, tamanyo);
        if(i == totalTrozos){
            free(buffer);
            // bucle en el que el padre espera a los hijos.
            esperarHijos(totalTrozos);
        }
    }
}
```

crearTrozos(entrada, nombre, totalTrozos, buffer, tamanyo)

Tiene un bucle que se repite una vez por trozo. En cada vuelta crea una tubería con pipe() y hace fork(). El padre llama a escribirTrozoEnTuberia() y sigue con la siguiente vuelta. El hijo llama a ejecutarHijo() y sale del bucle con break para no crear más hijos. Devuelve i, que vale totalTrozos en el padre y menos en los hijos, y así partir() sabe quién es quién.

Llamadas: pipe(), fork().

```c
// Devuelve i: totalTrozos en el padre, y menor que totalTrozos en un hijo.
int crearTrozos(int entrada, char nombre[], int totalTrozos, char *buffer, int tamanyo){
    int i;
    int canal[2]; // = {lectura, escritura}

    for(i = 0; i < totalTrozos; i++){
        pipe(canal);
        if(fork() != 0){
            escribirTrozoEnTuberia(entrada, canal, buffer, tamanyo);
        }
        else{
            ejecutarHijo(canal, nombre, i, buffer, tamanyo);
            break;
        }
    }
    return i;
}
```

escribirTrozoEnTuberia(entrada, canal, buffer, tamanyo)

Solo la ejecuta el padre. Lee tamanyo bytes del fichero original con read() y los escribe en la tubería con write(). Se usa bytesLeidos y no tamanyo en el write porque el último trozo puede tener menos bytes.

Llamadas: read(), write().

```
void escribirTrozoEnTuberia(int entrada, int canal[], char *buffer, int tamanyo){
    int bytesLeidos;

    bytesLeidos = read(entrada, buffer, tamanyo);
    write(canal[1], buffer, bytesLeidos);
}
```

prepararNombreTrozo(ficheroTrozo, nombre, i)

Construye el nombre del fichero del trozo con sprintf. Si i es menor que 10 pone un 0 delante (.h00, .h01...), y si no, no (.h10, .h11...).

Llamadas: sprintf().

```
void prepararNombreTrozo(char *ficheroTrozo, char *nombre, int i){

    if(i < 10){
        sprintf(ficheroTrozo, "%s.h0%d", nombre, i);
    }
    else{
        sprintf(ficheroTrozo, "%s.h%d", nombre, i);
    }
}
```

ejecutarHijo(canal, nombre, i, buffer, tamanyo)

Solo la ejecuta el hijo. Prepara el nombre de su fichero y lo crea con creat(). Después hay un bucle que lee de la tubería y escribe en el fichero lo que va llegando. totalLeidos cuenta los bytes ya escritos. El bucle acaba cuando se han leído tamanyo bytes o cuando read devuelve 0 o -1, que ocurre en el último trozo. Al final cierra el fichero y libera el buffer.

Llamadas: creat(), read(), write(), close(), free().

```
void ejecutarHijo(int canal[], char *nombre, int i, char *buffer, int tamanyo){
    int salida;
    int bytesLeidos;
    int totalLeidos = 0;
    char ficheroTrozo[50];

    prepararNombreTrozo(ficheroTrozo, nombre, i);
    salida = creat(ficheroTrozo, 0666);

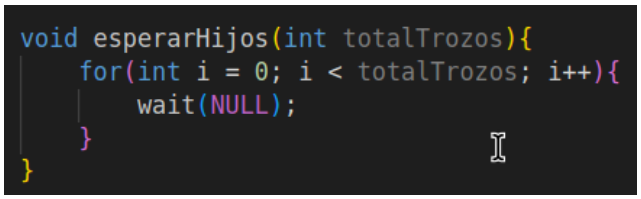

    do {
        bytesLeidos = read(canal[0], buffer, tamanyo);
        if (bytesLeidos > 0) {
            write(salida, buffer, bytesLeidos);
            totalLeidos += bytesLeidos;
        }
    } while (bytesLeidos > 0 && totalLeidos < tamanyo);

    close(salida);
    free(buffer);
}
```

esperarHijos(totalTrozos)

Hace wait(NULL) tantas veces como hijos se han creado, para que el padre no termine antes que ellos.

Llamadas: wait().



```
void esperarHijos(int totalTrozos){
    for(int i = 0; i < totalTrozos; i++){
        wait(NULL);
    }
}
```

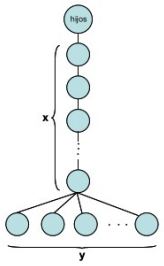
4. Ejercicio hijos.c

4.1. Enunciado y salida esperada

Ejercicio 3. Comunicación entre procesos: memoria compartida. (**3 puntos**)

Diseña un programa llamado **hijos.c** que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros

```
$hijos x y
```



El proceso **hijos** debe mostrar el siguiente mensaje:

"Soy el superpadre (pid) : mis hijos finales son: `pidx1, pidx2, pidx3, …, pidxy`"

Los procesos de la parte baja deben mostrar el siguiente mensaje**:**

"Soy el subhijo pidx1, mi padres son: `pid1, pid2, …, pidx`"

4.2. Tareas y justificación

- **Comprobar argumentos**: hacen falta dos (x e y). Si no, se muestra el uso y se sale.
- **Crear memoria compartida**: Para ello se utilizan dos vectores y necesitamos almacenar el pid inicial de cada uno de ellos. Son imprescindibles crearlos para la memoria compartida, ya que un vector normal no sería de gran ayuda.
- **Crear la cadena**: un bucle similar a ejec.c donde el padre espera y el hijo sigue creando el siguiente.
- **Imprimir antepasados de cada hijo**: cuando vuelve la cadena, cada hijo lee vx e imprime los pid de sus antepasados.
- **Crear los hijos finales**: el último de la cadena crea y hijos que guardan su pid en vy y esperan 10 segundos con alarm() y pause().
- **Esperar a los hijos**: cada padre hace espera para no morir antes que sus hijos.
- **Imprimir los hijos finales**: cuando vuelve la cadena, el super padre lee vy e imprime los pid.
- **Liberar la memoria**: cada proceso se desconecta de la memoria y el super padre la borra del sistema.

4.3. Análisis del código

main()

Comprueba que argc sea 3 y, si no, imprime el uso y termina. Guarda el pid del super padre en pid_sp, convierte x e y con atoi() y crea y conecta las dos memorias compartidas. Después llama a crearCadenaHijos() y, según el valor de i que devuelve, decide qué hace cada proceso:

- i == 1: es el super padre, imprime los hijos finales con imprimirSuperPadre().
- i == x + 1: es el último de la cadena, crea los hijos finales con crearHijosFinales().
- Cualquier otro valor: es un proceso intermedio y no hace nada más.

Al final, todos los procesos desconectan la memoria con desconectarMemoria(). Después, solo el super padre (i == 1) la borra con borrarMemoria().

Llamadas: getpid(), atoi().

```c
int main(int argc, char *argv[]) {
    int i, x, y, vectorX, vectorY;
    int *vx, *vy;

    if (argc != 3) {
        printf("Error, usage: ./hijos x y\n");
        return 0;
    }

    pid_sp = getpid();
    x = atoi(argv[1]);
    y = atoi(argv[2]);

    vectorX = crearMemoria(x);
    vx = conectarMemoria(vectorX);

    vectorY = crearMemoria(y);
    vy = conectarMemoria(vectorY);

    i = crearCadenaHijos(x, vx);

    if (i == 1) {
        imprimirSuperPadre(y, vy);
    }
    else {
        if (i == x + 1) {
        crearHijosFinales(y, vy, x, vx);
        }
    }

    desconectarMemoria(vx);
```

```c
    desconectarMemoria(vy);

    // solo el super padre la borra, porque es el último en terminar
    if (i == 1) {
        borrarMemoria(vectorX);
        borrarMemoria(vectorY);
    }

    return 0;
```

crearMemoria(elementos)

Reserva un segmento de memoria compartida para elementos enteros con shmget(). Usa IPC_PRIVATE y permisos 0666. Devuelve el identificador del segmento.

Llamadas: shmget().

conectarMemoria(vector)

Recibe el identificador y lo conecta al espacio del proceso con shmat(). Devuelve un puntero a int, que se usa como un vector normal.

Llamadas: shmat().

desconectarMemoria(vector)

Recibe el puntero que devolvió conectarMemoria() y lo desconecta del proceso con shmdt(). La memoria sigue existiendo en el sistema y los demás procesos la siguen viendo. Cada proceso que la conectó (y como los hijos la heredan con el fork(), son todos) tiene que desconectarla.

Llamadas: shmdt().

borrarMemoria(vector)

Recibe el identificador del segmento y lo marca para borrar con shmctl() y IPC_RMID. El sistema lo elimina cuando ya no queda ningún proceso conectado. Se hace una sola vez por segmento, no en cada proceso. Esta función solo la llama el super padre, porque es el último en terminar.

Llamadas: shmctl().

```c
int crearMemoria(int elementos) {
    return shmget(IPC_PRIVATE, sizeof(int) * elementos, IPC_CREAT | 0666);
}

int *conectarMemoria(int vector) {
    return (int *) shmat(vector, 0, 0);
}

void desconectarMemoria(int *vector) {
    shmdt(vector);
}

void borrarMemoria(int vector) {
    shmctl(vector, IPC_RMID, NULL);
}
```

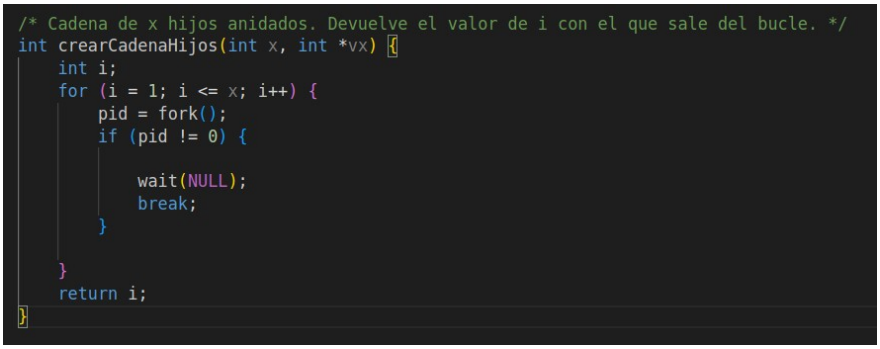
despierta()

Función vacía. Solo sirve para que pause() se desbloquee cuando llega SIGALRM.

crearCadenaHijos(x, vx)

Es un bucle de 1 a x con fork(). Si es el padre, hace wait(NULL) para esperar a su hijo y sale con break. Si es el hijo, guarda su pid en vx[i-1], imprime sus antepasados con imprimirAncestros() y sigue a la siguiente vuelta para crear a su propio hijo. Devuelve i: 1 en el super padre (su primer fork es el que hace break), valores intermedios en los demás y x + 1 en el último, que termina el bucle sin hacer break.

Llamadas: fork(), wait(), getpid().



```c
/* Cadena de x hijos anidados. Devuelve el valor de i con el que sale del bucle. */
int crearCadenaHijos(int x, int *vx) {
    int i;
    for (i = 1; i <= x; i++) {
        pid = fork();
        if (pid != 0) {

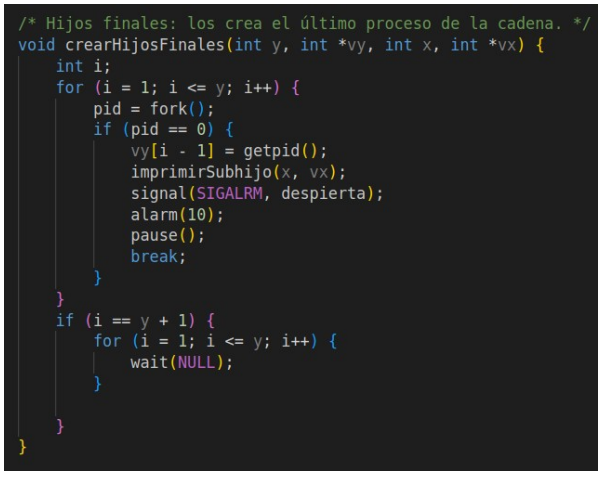
            wait(NULL);
            break;
        }

    }
    return i;
}
```

crearHijosFinales(y, vy,x,vx)

Solo la ejecuta el último de la cadena. Bucle de 1 a y con fork(). Cada hijo guarda su pid en vy, instala despierta para SIGALRM, hace alarm(10) y se queda en pause(). Cuando despierta sale con break. El padre sigue el bucle y, cuando ha creado todos (i == y + 1), espera a los y con wait(). La x y la vx se necesitan para llamar a la función de imprimirSubhijo.

Llamadas: fork(), getpid(), signal(), alarm(), pause(), wait().



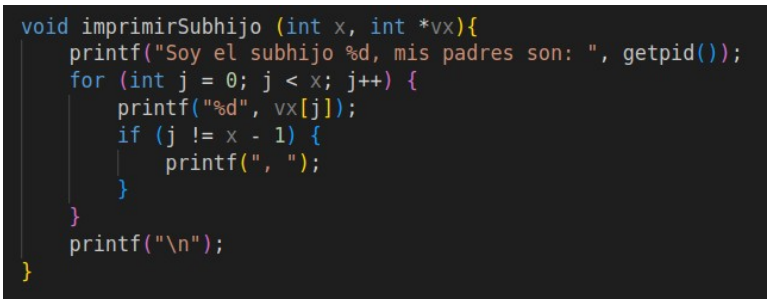
```c
/* Hijos finales: los crea el último proceso de la cadena. */
void crearHijosFinales(int y, int *vy, int x, int *vx) {
    int i;
    for (i = 1; i <= y; i++) {
        pid = fork();
        if (pid == 0) {
            vy[i - 1] = getpid();
            imprimirSubhijo(x, vx);
            signal(SIGALRM, despierta);
            alarm(10);
            pause();
            break;
        }
    }
    if (i == y + 1) {
        for (i = 1; i <= y; i++) {
            wait(NULL);
        }

    }
}
```

I

mprimir Subhijo(x,vx)

Imprime “Soy el subhijo “ con su pid y con los pids de sus ancestros, recorriendo un fork de x para sacar a dichos padres, abuelos,etc.

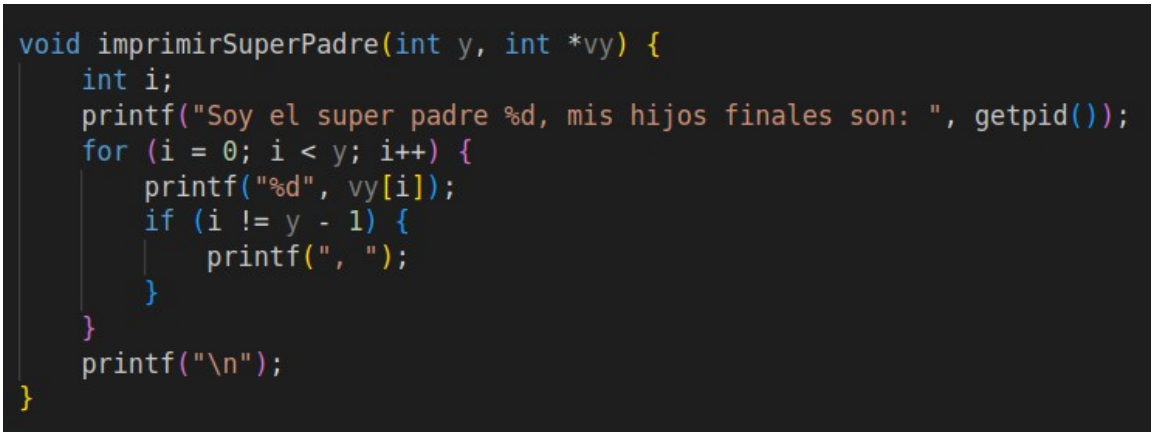


```c
void imprimirSubhijo (int x, int *vx){
    printf("Soy el subhijo %d, mis padres son: ", getpid());
    for (int j = 0; j < x; j++) {
        printf("%d", vx[j]);
        if (j != x - 1) {
            printf(", ");
        }
    }
    printf("\n");
}
```

ImprimirSuperPadre(y, vy)

Imprime “Soy el super padre” con su pid y recorre vy escribiendo los pid de los hijos finales separados por comas, sin coma al final.

Llamadas: getpid(), printf().



```c
void imprimirSuperPadre(int y, int *vy) {
    int i;
    printf("Soy el super padre %d, mis hijos finales son: ", getpid());
    for (i = 0; i < y; i++) {
        printf("%d", vy[i]);
        if (i != y - 1) {
            printf(", ");
        }
    }
    printf("\n");
}
```